

cheatsheet_openmp_exemplos

Cheat Sheet — OpenMP: Exemplos de Código

Complemento do cheat sheet conceitual. Cada exemplo é mínimo e compilável com `gcc -fopenmp arquivo.c -o programa`.

1. Região Paralela Básica

`#pragma omp parallel`

```
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        printf("Alo da thread %d\n", omp_get_thread_num());
    }
    return 0;
}
// Com 4 threads: imprime 4 linhas (uma por thread), ordem não garantida.
```

`num_threads(N)`

```
#pragma omp parallel num_threads(4)
{
    printf("Thread %d de %d\n", omp_get_thread_num(),
        omp_get_num_threads());
}
```

`if(expr)`

```
int n = 2;
#pragma omp parallel if(n > 4) num_threads(4)
{
    // Se n <= 4, roda sequencial (só 1 thread executa o bloco)
    printf("Thread %d\n", omp_get_thread_num());
}
```

`default(none)` — força escopar tudo manualmente

```
int a = 10, b = 20;
#pragma omp parallel default(none) shared(a) private(b)
{
    b = omp_get_thread_num(); // 'a' e 'b' precisam estar listados, senão
    erro de compilação
    printf("a=%d b=%d\n", a, b);
}
```

2. Worksharing

#pragma omp for

```
#pragma omp parallel num_threads(4)
{
    #pragma omp for
    for (int i = 0; i < 8; i++) {
        printf("i=%d thread=%d\n", i, omp_get_thread_num());
    }
}
// Cada i roda uma única vez, dividido entre as threads.
```

#pragma omp parallel for (forma combinada)

```
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 8; i++) {
    printf("i=%d thread=%d\n", i, omp_get_thread_num());
}
```

#pragma omp single

```
#pragma omp parallel num_threads(4)
{
    #pragma omp single
    {
        printf("Só uma thread (a %d) faz isso\n", omp_get_thread_num());
    }
    printf("Todas as threads chegam aqui depois da barreira\n");
}
```

#pragma omp master

```
#pragma omp parallel num_threads(4)
{
```

```
#pragma omp master
{
    printf("Só a thread 0 faz isso, sem barreira\n");
}
printf("Threads não esperam o master terminar\n");
}
```

#pragma omp sections / section

```
#pragma omp parallel num_threads(2)
{
    #pragma omp sections
    {
        #pragma omp section
        {
            printf("Tarefa A pela thread %d\n", omp_get_thread_num());
        }
        #pragma omp section
        {
            printf("Tarefa B pela thread %d\n", omp_get_thread_num());
        }
    }
}
// Cada 'section' roda em uma thread diferente (se houver threads
suficientes).
```

3. Cláusulas de for / parallel for

collapse(n)

```
#pragma omp parallel for collapse(2) num_threads(4)
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 10; j++) {
        printf("tid=%d i=%d j=%d\n", omp_get_thread_num(), i, j);
    }
}
// As 30 iterações (3x10) são tratadas como um único espaço de 30 e
divididas entre threads.
```

ordered

```
#pragma omp parallel for ordered schedule(static,1) num_threads(4)
for (int i = 0; i < 10; i++) {
    printf("[Thread %d] executou iteracao %d\n", omp_get_thread_num(), i);
    #pragma omp ordered
}
```

```

    {
        printf(">>> Saida ordenada: iteracao %d\n", i);
    }
}
// O bloco 'ordered' sempre imprime na ordem 0,1,2...9, mesmo que
// as iterações rodem fora de ordem entre as threads.

```

nowait

```

#pragma omp parallel num_threads(4)
{
    #pragma omp for nowait
    for (int i = 0; i < 100; i++) { /* trabalho A */ }

    // Sem 'nowait', as threads esperariam TODAS terminarem o laço acima
    // antes de seguir. Com 'nowait', cada thread segue direto:
    #pragma omp for
    for (int i = 0; i < 100; i++) { /* trabalho B, independente do A */ }
}

```

4. Escopo de Dados

shared (padrão para variáveis externas)

```

int contador = 0;
#pragma omp parallel shared(contador) num_threads(4)
{
    #pragma omp critical
    contador++; // protegido, senão haveria condição de corrida
}
printf("contador final = %d\n", contador); // sempre 4

```

private

```

int x = 100;
#pragma omp parallel private(x) num_threads(4)
{
    x = omp_get_thread_num(); // x é uma cópia local, começa indefinida
    printf("dentro: x=%d\n", x);
}
printf("fora: x=%d\n", x); // ainda é 100, private não altera o valor
externo

```

firstprivate

```

int x = 100;
#pragma omp parallel firstprivate(x) num_threads(4)
{
    x += omp_get_thread_num(); // cada thread começa com x=100, e soma seu
    tid
    printf("thread %d: x=%d\n", omp_get_thread_num(), x); // 100,101,102,103
}
printf("fora: x=%d\n", x); // continua 100

```

lastprivate

```

int ultimo;
#pragma omp parallel for lastprivate(ultimo) num_threads(4)
for (int i = 0; i < 8; i++) {
    ultimo = i; // cada thread tem sua cópia
}
printf("ultimo = %d\n", ultimo); // recebe o valor da ÚLTIMA iteração lógica
(i=7)

```

5. schedule

```

#include <stdio.h>
#include <omp.h>

int main() {
    // static sem chunk: blocos de tamanho ~N/threads
    #pragma omp parallel for schedule(static) num_threads(4)
    for (int i = 0; i < 16; i++) printf("[static] i=%d tid=%d\n", i,
    omp_get_thread_num());

    // static com chunk=1: round-robin, uma iteração por vez
    #pragma omp parallel for schedule(static, 1) num_threads(4)
    for (int i = 0; i < 16; i++) printf("[static,1] i=%d tid=%d\n", i,
    omp_get_thread_num());

    // dynamic: threads pedem trabalho conforme terminam
    #pragma omp parallel for schedule(dynamic, 3) num_threads(4)
    for (int i = 0; i < 16; i++) printf("[dynamic,3] i=%d tid=%d\n", i,
    omp_get_thread_num());

    // guided: blocos grandes no início, menores no fim
    #pragma omp parallel for schedule(guided, 2) num_threads(4)
    for (int i = 0; i < 16; i++) printf("[guided,2] i=%d tid=%d\n", i,
    omp_get_thread_num());
}

```

```
// runtime: usa a variável de ambiente OMP_SCHEDULE
#pragma omp parallel for schedule(runtime) num_threads(4)
for (int i = 0; i < 16; i++) printf("[runtime] i=%d tid=%d\n", i,
omp_get_thread_num());

return 0;
}
// Testar runtime assim: OMP_SCHEDULE="dynamic,2" ./programa
```

6. Sincronização

critical

```
int soma = 0;
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 1000; i++) {
    #pragma omp critical
    soma += i; // só uma thread por vez executa esta linha
}
printf("soma = %d\n", soma);
```

critical(nome) — cadeados nomeados independentes

```
int a = 0, b = 0;
#pragma omp parallel num_threads(4)
{
    #pragma omp critical(lockA)
    a++; // protegido pelo cadeado "lockA"

    #pragma omp critical(lockB)
    b++; // protegido por um cadeado DIFERENTE, "lockB"
    // threads podem mexer em 'a' e 'b' simultaneamente sem se bloquear
    entre si
}
```

atomic update (padrão)

```
int contador = 0;
#pragma omp parallel for num_threads(4)
for (int i = 0; i < 1000; i++) {
    #pragma omp atomic
    contador++; // operação atômica simples, mais rápida que critical
}
```

atomic read / atomic write

```
int flag = 0, leitura;
#pragma omp parallel num_threads(2)
{
    if (omp_get_thread_num() == 0) {
        #pragma omp atomic write
        flag = 1;
    } else {
        #pragma omp atomic read
        leitura = flag;
    }
}
```

atomic capture

```
int x = 0, z;
#pragma omp parallel num_threads(4)
{
    #pragma omp atomic capture
    z = x++; // lê o valor antigo de x E incrementa, tudo atomicamente
}
```

barrier

```
#pragma omp parallel num_threads(4)
{
    printf("Thread %d antes da barreira\n", omp_get_thread_num());
    #pragma omp barrier
    printf("Thread %d depois da barreira\n", omp_get_thread_num());
    // NENHUMA thread imprime "depois" antes de TODAS imprimirem "antes"
}
```

reduction

```
double soma = 0.0;
#pragma omp parallel for reduction(+:soma) num_threads(4)
for (int i = 0; i < 1000000; i++) {
    soma += 1.0;
}
printf("soma = %.0f\n", soma); // 1000000, sem condição de corrida

// outros operadores possíveis: reduction(*:prod), reduction(max:maior),
reduction(min:menor)
```

7. Tasks

task básica

```
#pragma omp parallel num_threads(4)
#pragma omp single
{
    #pragma omp task
    printf("Tarefa A\n");
    #pragma omp task
    printf("Tarefa B\n");
    // A e B podem executar em qualquer ordem, em threads diferentes
}
```

taskwait

```
#pragma omp parallel num_threads(2)
#pragma omp single
{
    printf("A ");
    #pragma omp task
    printf("race ");
    #pragma omp task
    printf("car ");
    #pragma omp taskwait // espera as duas tasks acima terminarem
    printf("is fun to watch\n");
}
```

if(expr) em task

```
void trabalho_pesado(int n) { /* ... */ }

#pragma omp parallel num_threads(4)
#pragma omp single
{
    for (int i = 0; i < 100; i++) {
        // se n for pequeno, executa sequencial (evita overhead de criar
task)
        #pragma omp task if(i > 10)
        trabalho_pesado(i);
    }
}
```

final(expr)


```

int fib_seq(int n) { return n < 2 ? n : fib_seq(n-1) + fib_seq(n-2); }

int fib(int n) {
    if (n < 2) return n;
    int x, y;
    #pragma omp task shared(x) final(n <= 10)
    x = (n <= 10) ? fib_seq(n-1) : fib(n-1); // corta recursão de tasks
    quando n fica pequeno
    #pragma omp task shared(y) final(n <= 10)
    y = (n <= 10) ? fib_seq(n-2) : fib(n-2);
    #pragma omp taskwait
    return x + y;
}

```

untied

```

#pragma omp parallel num_threads(4)
#pragma omp single
{
    #pragma omp task untied
    {
        // esta tarefa pode ser retomada por QUALQUER thread do time,
        // não necessariamente a que começou a executá-la
        printf("Tarefa untied rodando\n");
    }
}

```

priority(valor)

```

#pragma omp parallel num_threads(4)
#pragma omp single
{
    #pragma omp task priority(10)
    printf("Tarefa de alta prioridade\n");
    #pragma omp task priority(1)
    printf("Tarefa de baixa prioridade\n");
}

```

shared / firstprivate em task (exemplo Fibonacci)

```

int fib(int n) {
    if (n < 2) return n;
    int x, y;
    #pragma omp task shared(x) // x precisa ser shared para o resultado
    "voltar"
    { x = fib(n - 1); }
}

```

```

    #pragma omp task shared(y)
    { y = fib(n - 2); }
    #pragma omp taskwait
    return x + y;
}

int main() {
    int N = 10;
    #pragma omp parallel
    #pragma omp single
    {
        printf("fib(%d) = %d\n", N, fib(N));
    }
}

```

depend(in/out/inout)

```

int x = 0;
#pragma omp parallel num_threads(2)
#pragma omp single
{
    #pragma omp task depend(out: x)
    { x = 10; printf("T1: escreveu x=%d\n", x); }           // produz x

    #pragma omp task depend(in: x)
    { printf("T2: leu x=%d\n", x); }                       // só roda depois
de T1

    #pragma omp task depend(in: x)
    { printf("T3: leu x=%d\n", x); }                       // também depende
de T1, mas

                                                                // pode rodar em
paralelo com T2

    #pragma omp task depend(inout: x)
    { x++; printf("T4: atualizou x=%d\n", x); }            // espera T1, T2 e
T3
}

```

taskgroup

```

#pragma omp parallel num_threads(4)
#pragma omp single
{
    #pragma omp taskgroup
    {
        #pragma omp task
        {

```

```

        printf("Tarefa pai\n");
        #pragma omp task
        printf("Tarefa filha (neta do single)\n");
    }
} // espera a tarefa pai E a tarefa filha (descendente) terminarem aqui
printf("Só chega aqui depois de TODA a árvore de tasks terminar\n");
}

```

taskyield

```

#pragma omp parallel num_threads(2)
#pragma omp single
{
    #pragma omp task
    {
        printf("Início da tarefa\n");
        #pragma omp taskyield // permite que a thread execute outra tarefa
        aqui, se quiser
        printf("Fim da tarefa\n");
    }
}

```

taskloop

```

double a[1000000], b[1000000], c[1000000];
// (supondo a e b já preenchidos)

#pragma omp parallel num_threads(4)
#pragma omp single
{
    #pragma omp taskloop grainsize(1000)
    for (int i = 0; i < 1000000; i++) {
        c[i] = a[i] + b[i];
    }
}
// Divide o laço em blocos de ~1000 a 2000 iterações, cada bloco vira uma
task.

```

taskloop com num_tasks

```

#pragma omp parallel num_threads(4)
#pragma omp single
{
    #pragma omp taskloop num_tasks(8)
    for (int i = 0; i < 1000000; i++) {
        c[i] = a[i] + b[i];
    }
}

```

```
}  
// Cria exatamente 8 tarefas, cada uma cobrindo ~125000 iterações.
```

8. Funções de Biblioteca

```
#include <stdio.h>  
#include <omp.h>  
  
int main() {  
    printf("Núcleos disponíveis: %d\n", omp_get_num_procs());  
  
    omp_set_num_threads(4);  
  
    double inicio = omp_get_wtime();  
  
    #pragma omp parallel  
    {  
        if (omp_get_thread_num() == 0) {  
            printf("Threads no time: %d\n", omp_get_num_threads());  
        }  
        printf("Estou em região paralela? %d\n", omp_in_parallel());  
    }  
  
    double fim = omp_get_wtime();  
    printf("Tempo: %f segundos\n", fim - inicio);  
  
    return 0;  
}
```

Locks manuais (`omp_init_lock` , `omp_set_lock` , `omp_unset_lock`)

```
#include <omp.h>  
#include <stdio.h>  
  
int main() {  
    omp_lock_t trava;  
    omp_init_lock(&trava);  
    int contador = 0;  
  
    #pragma omp parallel num_threads(4)  
    {  
        omp_set_lock(&trava);  
        contador++; // seção crítica manual, alternativa a 'critical'  
        omp_unset_lock(&trava);  
    }  
}
```

```
    printf("contador = %d\n", contador);  
    return 0;  
}
```

9. Variáveis de Ambiente (uso no terminal)

```
# Define número de threads padrão  
export OMP_NUM_THREADS=4  
./programa  
  
# Define schedule usado por schedule(runtime) no código  
export OMP_SCHEDULE="dynamic,4"  
./programa  
  
# Define tamanho de pilha por thread  
export OMP_STACKSIZE=16M  
./programa  
  
# Controla afinidade de threads a núcleos físicos  
export OMP_PROC_BIND=true  
export OMP_PLACES=cores  
./programa
```